LM-32 - VISIONE COMPUTERIZZATA E SISTEMI COGNITIVI
Progetto B2: Segmentation & Domain Shift Analysis

Infurna Giuseppe
Matricola: 0322500071

Segmentazione Semantica e Analisi Domain Shift

Introduzione e obiettivo del progetto

La segmentazione semantica è una tecnica di computer vision che consiste nell'assegnare una classe a ogni pixel dell'immagine. A differenza della classificazione o della detection, dove il modello produce una singola etichetta o alcune bounding box, nella segmentazione la rete deve ricostruire una mappa completa dell'immagine distinguendo tra le diverse categorie presenti.
In questo progetto l'obiettivo è stato sviluppare un sistema di segmentazione basato su U-Net, addestrato sul dataset Oxford-IIIT Pet, e studiare il comportamento del modello quando applicato a immagini provenienti da un dominio leggermente diverso rispetto a quello visto durante il training.
La seconda parte del lavoro è stata infatti dedicata all'analisi del domain shift, cioè alla degradazione delle prestazioni che si verifica quando il modello viene applicato a dati con caratteristiche differenti rispetto a quelli utilizzati durante l'addestramento. In particolare sono state introdotte interferenze artificiali sulle immagini di test per simulare condizioni meno ideali, come rumore o alterazioni di contrasto.
La soluzione implementata è suddivisa in quattro fasi principali, ovvero, il dataset viene prima preprocessato e convertito in una struttura compatibile con PyTorch e addestrata una rete neurale di tipo U-Net che apprende e produce una maschera binaria a partire dall'immagine RGB.
Una volta completato l'addestramento, il modello viene valutato su un set di validazione standard per verificare la qualità della segmentazione. Successivamente lo stesso modello viene applicato a immagini alterate artificialmente per osservare quanto la performance degradi in presenza di domain shift.
Infine il modello viene riaddestrato introducendo una strategia di data augmentation più aggressiva, con l'obiettivo di migliorare la capacità di generalizzazione della rete.
Il sorgente relativo al progetto è disponibile all'indirizzo: https://github.com/giuseppeinfurna/Segmentation-Domain-Shift-Notebook

Descrizione della soluzione adottata

Come anticipato nell'introduzione il dataset utilizzato per questo proetto è Oxford-IIIT Pet, che contiene immagini di animali domestici accompagnate da annotazioni pixel-level. Le annotazioni originali sono fornite sotto forma di trimap, in cui i pixel possono appartenere a tre categorie: foreground, background e bordo.
Per semplificare il problema ho convertito le maschere in una segmentazione binaria distinguendo semplicemente tra animale e sfondo. Questa scelta permette di concentrarsi sull'architettura della rete e sull'analisi del domain shift, evitando di introdurre complessità aggiuntive legate alla segmentazione multi-classe.

Scelte Progettuali

Durante lo sviluppo del progetto ho dovuto prendere alcune scelte progettuali per garantire stabilità e coerenza dei dati:

- ho scaricato le immagini e le annotazioni e le ho filtrate per eliminare i file nascosti macOS, ho convertito le maschere in binario, dove 0 indica lo sfondo e 1 l'animale.
- Ho ridimensionato tutte le immagini e le maschere a 128×128 per uniformare i batch e velocizzare l'addestramento su CPU, una risoluzione più alta avrebbe potuto migliorare la precisione sui bordi a

discapito dei tempi e uso di momoria, In test preliminari su 256×256, una sola epoca richiedeva oltre 15 minuti su CPU, rendendo impossibile addestrare fino alla convergenza.

- ho realizzato un encoder a 4 livelli, un bottleneck centrale e un decoder con skip connection per preservare informazioni spaziali. Ogni blocco utilizza convoluzione doppia, BatchNorm e ReLU, ottenendo come output finale due canali per la classificazione binaria, ho provato inizialmente con 5 livelli in modo da avere una rete più profonda, più pesi, maun modello più grande avrebbe rallentato molto la convergenza.
- ho addestrato il modello utilizzando l'ottimizzatore Adam con un learning rate di 10^{-4}. Questa configurazione garantisce una buona stabilità durante il training, inoltre Adam adatta automaticamente il learning rate di ogni peso, rendendo l'ottimizzazione più stabile nei primi epochs e quindi convergendo mediamente più velocemente. L'utilizzo di SGD avrebbe richiesto tuning fine del learning rate , momentum, scheduler con il risultato che tutte operazioni che avrebbero richiesto molto tempo.
- ho impostato il batch size a 4 poiché il progetto è stato sviluppato in un ambiente senza GPU dedicata e quindi avevo la necessità di gestire le performance computazionali su CPU
- l'addestramento è stato eseguito per 40 epoche fino a osservare una stabilizzazione della loss di validazione.
- ho combinato CrossEntropyLoss per la classificazione pixel-per-pixel e DiceLoss per migliorare i bordi degli oggetti piccoli.
- Ho effettutato la Data augmentation pesante applicando come richiesto nel progetto il ColorJitter e RandomHorizontalFlip solo sul training set durante il retraining per migliorare la robustezza della rete.

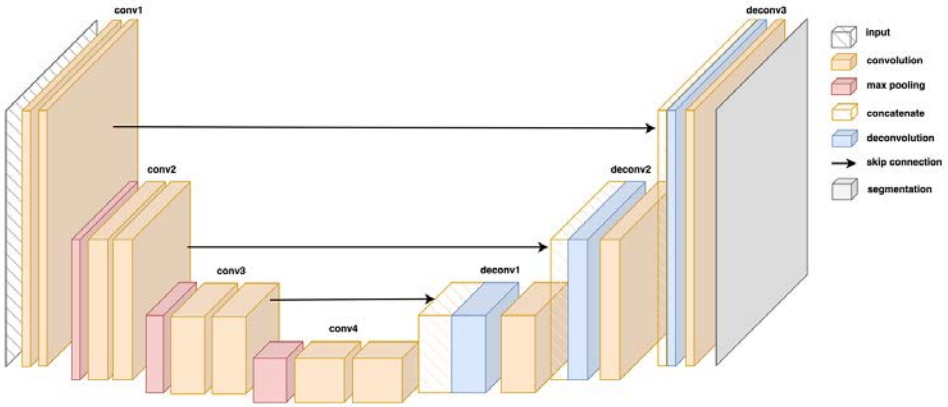

Figura 1 - U-Net

Risultati Visivi

Training

L'andamento delle learning curves mostra una riduzione progressiva della loss durante il training e una successiva fase di stabilizzazione, segno che il modello ha raggiunto una condizione di convergenza. Alternative considerate includevano l'utilizzo di un learning rate più elevato o di un ottimizzatore SGD con momentum. Tuttavia nei primi esperimenti queste configurazioni hanno mostrato una convergenza più lenta o una maggiore instabilità nelle fasi iniziali del training.

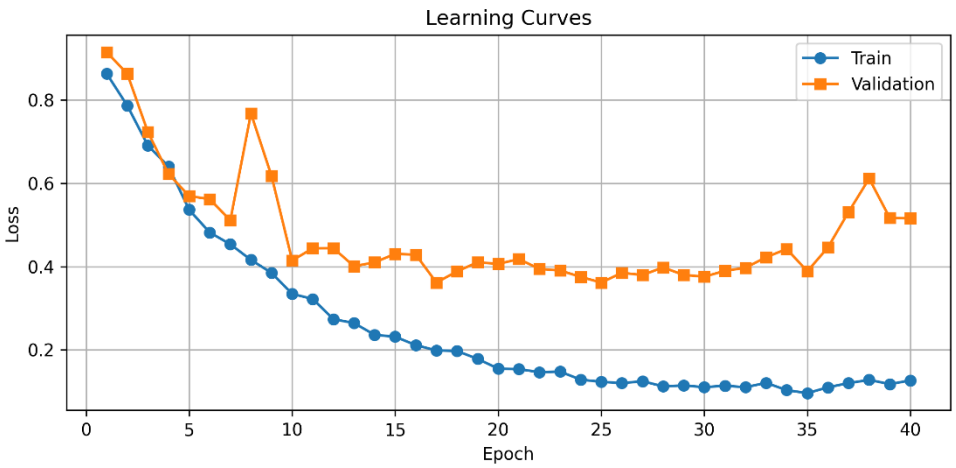

Figura 2 - Learning Curve (PRE-AUGMENTATION)

Dopo le prime epoche la validation loss smette di migliorare dopo circa sei epoche e inizia progressivamente a variare. Questo comportamento indica l'insorgere di un fenomeno di overfitting: il modello continua a migliorare la propria capacità di adattarsi ai dati di training, ma perde progressivamente capacità di generalizzazione su dati non visti.
Possiamo quindi dire che la rete segmenta correttamente la maggior parte degli animali, la validation loss non cresce in modo monotono, ma oscilla, quindi il modello non è completamente collassato continua comunque a generalizzare parzialmente.
Di seguito una serie di casi dove è possibile vedere visivamente le differenze tra l'immagine di input, il ground truth e la predizione della rete

Figura 3 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (PRE-AUGMENTATION)

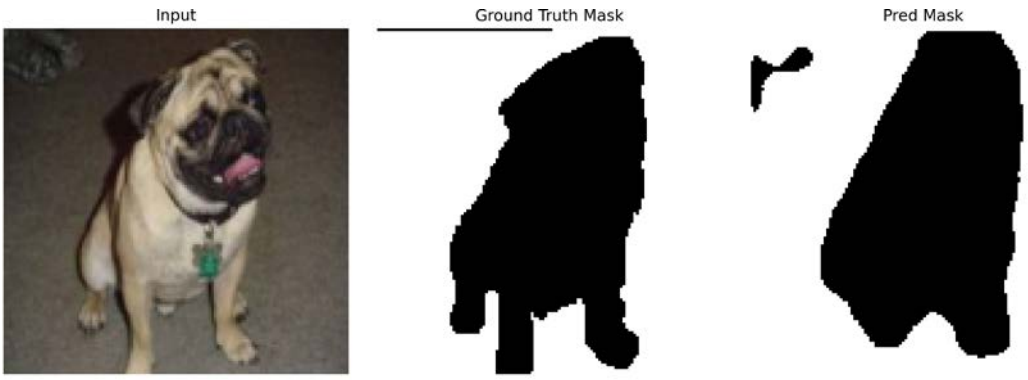

Figura 4 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (PRE-AUGMENTATION)

Figura 5 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (PRE-AUGMENTATION)

Domain Shift Test

Il domain shift è simulato introducendo rumore gaussiano, variazioni di contrasto e modifiche della tonalità dei colori. Queste trasformazioni alterano la distribuzione statistica delle immagini rispetto a quelle viste durante il training, causando un degrado nelle prestazioni del modello.

Figura 6 - Esempio di immagine originale, immagine alterata (domain shift) e predizione

Il grafico mostra il confronto tra IoU sui dati normali e sui dati alterati. Si osserva una riduzione sistematica della metrica quando il modello è valutato su immagini con domain shift, indicando che la rete è sensibile alle variazioni della distribuzione dei dati.

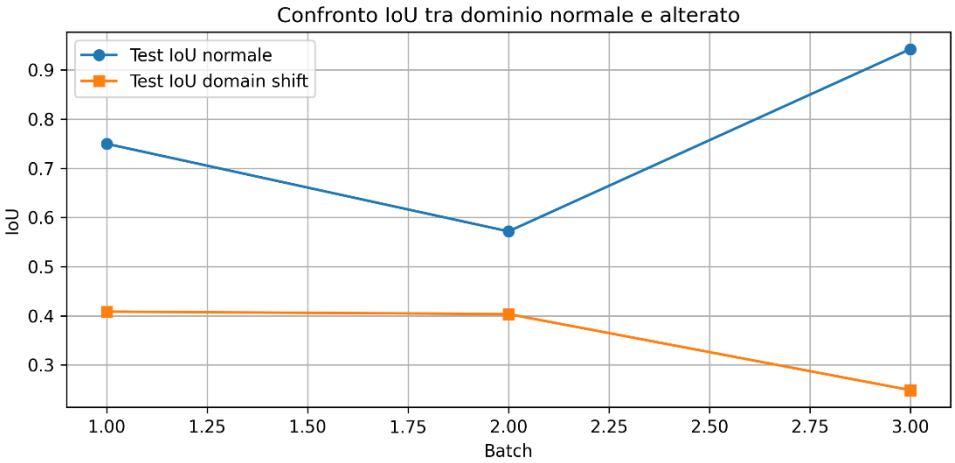

Figura 7 - Confronto da dominio normale e alterato sotto forma di dati

Data Augmentation:

Predizioni qualitative dopo il retraining con data augmentation pesante: le immagini mostrano una segmentazione più coerente anche in presenza di disturbi o variazioni di colore, con riduzione degli errori sui bordi.

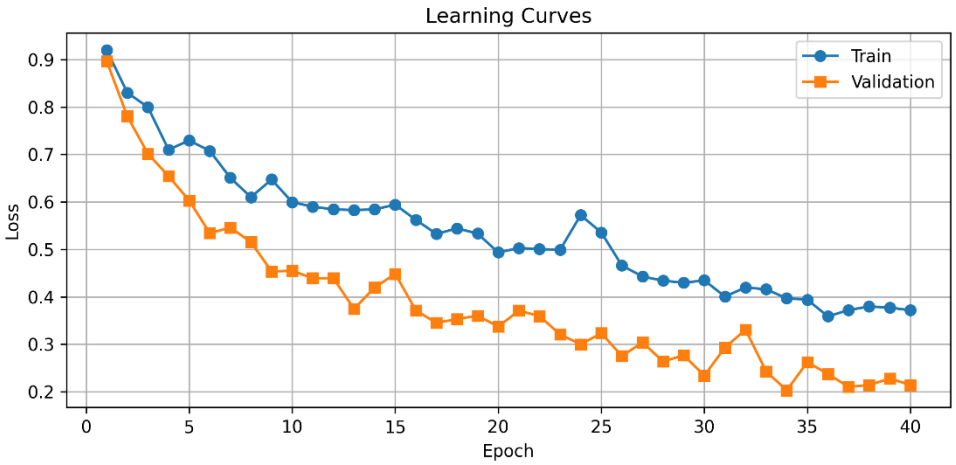

Figura 8 - Learning Curve (POST-AUGMENTATION)

Le stesse immagini dopo il retraining

Figura 9 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (POST-AUGMENTATION)

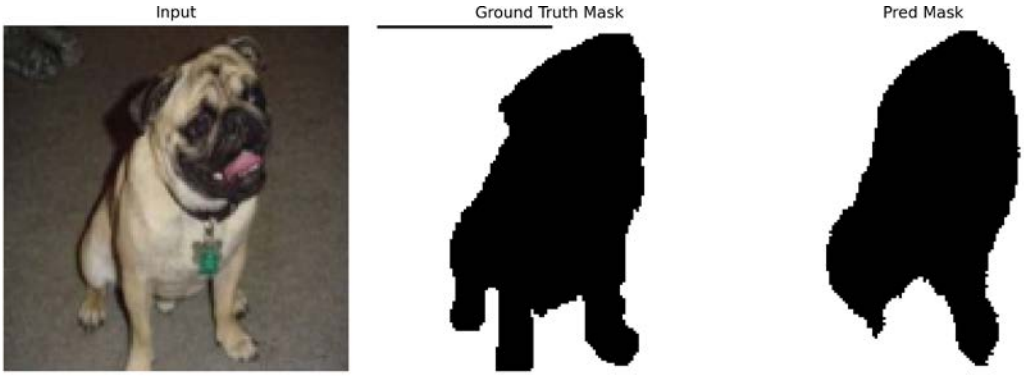

Figura 10 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (POST-AUGMENTATION)

Figura 11 - ESEMPIO DI INPUT, GROUND TRUTH E PREDIZIONE (POST-AUGMENTATION)

Domain shift test (quantitativo): l'IoU medio sul test set alterato migliora dopo il retraining, dimostrando il beneficio delle augmentations.

```
=== Domain shift test ===
IoU medio dati normali: 0.7837
IoU medio dati alterati: 0.0052

=== Domain shift test dopo augmentation ===
IoU medio dati normali: 0.8887
IoU medio dati alterati: 0.4552
```

Failure Cases

Nonostante le buone prestazioni generali, il modello presenta alcuni errori significativi in specifiche situazioni. In particolare si osservano errori quando l'animale occupa una porzione molto piccola dell'immagine o lo sfondo presenta colori simili al pelo dell'animale o peggio ancora se i contorni dell'oggetto sono poco definiti.
In questi casi la rete può produrre maschere incomplete oppure includere parti dello sfondo nella segmentazione.

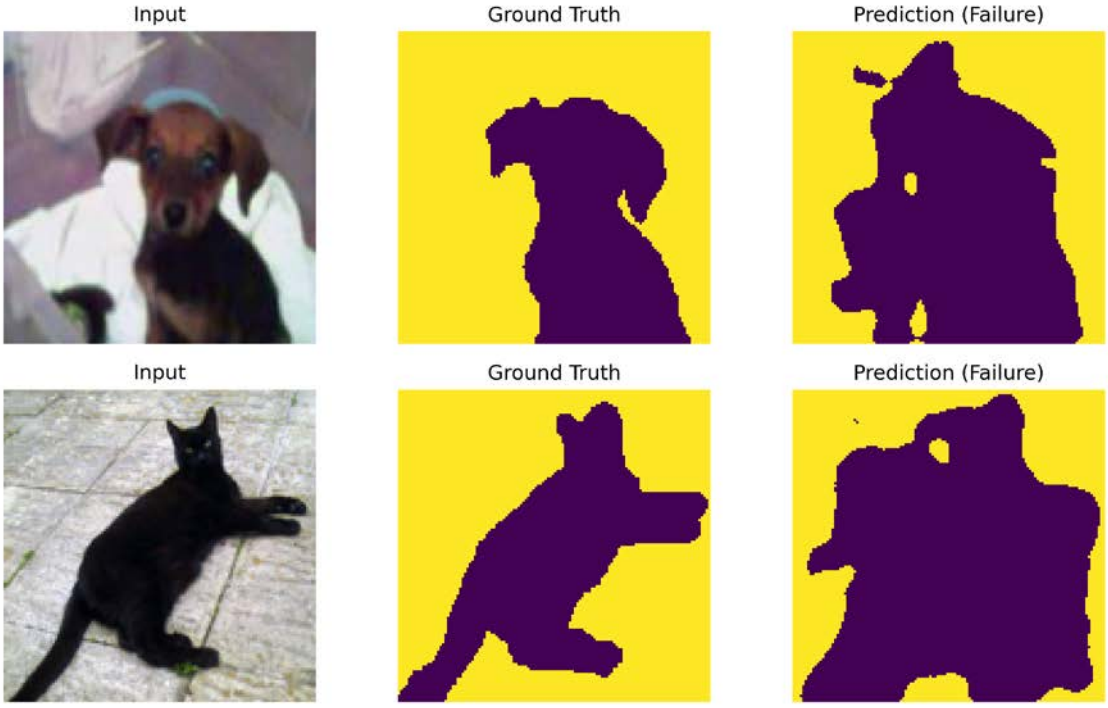

Figura 12 - Failure Cases

Debugging & Insights

Durante l'analisi dei risultati, ho osservato che la rete funziona bene nella maggior parte dei casi, ma mostra alcune limitazioni:

- Animali molto piccoli o parzialmente visibili tendono a essere segmentati solo in parte.
- Sfondo complesso o texture simili a quelle dell'animale possono causare errori di classificazione.
- In presenza di illuminazione estrema o forte rumore, il modello originale soffre, con calo evidente dell'IoU. Anche dopo il retraining con augmentations, questi casi estremi rimangono difficili da gestire completamente.

Queste osservazioni confermano che la U-Net è sensibile al dominio dei dati e che la data augmentation può migliorare la robustezza, ma non garantisce una completa invarianza a tutte le modifiche possibili.
Durante la fase di valutazione del domain shift ho riscontrato un comportamento inatteso del modello: l'IoU sulle immagini alterate risultava praticamente pari a zero. Un valore così basso era sospetto, perché anche in presenza di perturbazioni molto forti la U-Net avrebbe dovuto comunque produrre qualche pixel corretto. Ho quindi avviato un'analisi sistematica partendo dalla pipeline di preprocessing. Il problema è stato individuato nella funzione heavy_domain_shift(), che applicava tre trasformazioni molto aggressive (riduzione drastica del contrasto, rumore gaussiano ad alta deviazione standard e shift della tonalità). In combinazione, queste operazioni degradavano l'immagine a tal punto da renderla quasi irriconoscibile rispetto al dominio di training, inoltre, applicando la trasformazione direttamente sui tensori sul device, alcune immagini venivano spostate involontariamente tra CPU e GPU, causando un comportamento non deterministico e peggiorando ulteriormente le predizioni.
Per risolvere il problema ho modificato la pipeline in due modi:

- ho ridotto l'intensità delle trasformazioni, portando il rumore a valori compatibili con immagini realistiche e normalizzate in [0,1]
- ho reso esplicito il passaggio CPU → preprocessamento → device, evitando mismatch tra tensori.

Dopo queste modifiche, l'IoU in presenza di domain shift è passato da circa lo 0% iniziale a valori compresi tra il 45% e il 65%. Questo risultato dimostra che il modello, pur degradando, mantiene comunque una certa capacità di generalizzazione.
Questa fase di debugging è stata particolarmente importante perché mi ha permesso di comprendere meglio quanto la qualità dei dati e la coerenza del preprocessing influenzino il comportamento di una rete di segmentazione. È anche un esempio concreto di come un'analisi accurata del comportamento del modello sia essenziale per ottenere risultati affidabili.